

STAT 240

Week 7: Regression and Prediction

Owen G. Ward

Today

- See another regression method, K-Nearest Neighbors
- Consider how to compare the performance of these two regression methods, prediction tools in general

Logistics

- Midterm Solutions and grades posted in the next week or so
- I'll spend time going through common issues next week also

Quick Recap - Linear Regression

- Assume we have input $X = (X_1, \dots, X_n)$ and output $Y = (Y_1, \dots, Y_n)$ (example: X_i is month index, Y_i is demand for product A)
- We assume that the “true” demand is $f(X_i) = \alpha + \beta X_i$
- The “observed” demand is a noisy version of the “true” demand: $\alpha + \beta X_i + \epsilon_i$

$$\hat{\beta} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}, \quad \hat{\alpha} = \bar{Y} - \hat{\beta}\bar{X}. \quad (1)$$

Model Checking for Linear Regression

- Linear regression relies on some basic assumptions, that underlie the theory behind this model
- Assumes the true model we saw 1 slide ago
- Can we check those assumptions?
 - Yes, but not in this class (Stat 302/305)
 - We will focus on comparing this model using prediction

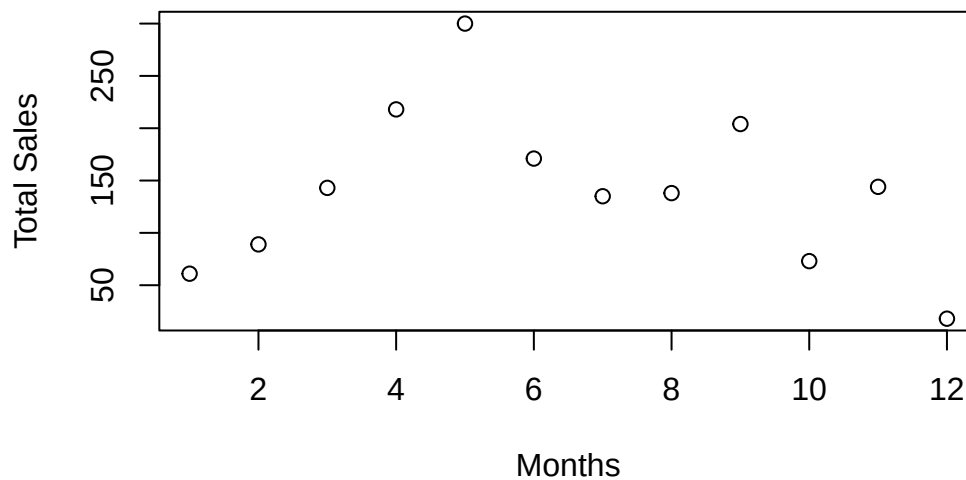
Back to original example

```
library(RSQLite)
db <- dbConnect(SQLite(), dbname = "xcoretail.sqlite")
dbListTables(db)
```

```
[1] "prices" "sales"
```

```
library(DBI)
months <- sprintf("2011-%02d", 1:12)
counts <- rep(NA, 12)
for(i in 1:length(counts)) {
  query <- sqlInterpolate(db, "SELECT SUM(Quantity) FROM sales
                                WHERE StockCode = '71053' AND
                                InvoiceDate LIKE ?month",
  month = paste0(months[i], "%") )
  request <- dbSendQuery(db, query)
  counts[i] <- as.numeric(dbFetch(request))
  dbClearResult(request)
}
```

```
plot(x = 1:12, y = counts, xlab = "Months", ylab = "Total Sales")
```



Predicting the next month

- Suppose it's October 1st and we want to predict how many sales there will be in the month
- Can we do this using linear regression?
 - Yes
- How do we decide if our model is any good?
 - Compare it to the true number of sales

Prediction

- Lets fit a linear regression to the first 9 months

```
x <- 1:9
y <- counts[1:9]

fit <- lm(y ~ x)
fit
```

Call:

```
lm(formula = y ~ x)
```

Coefficients:

(Intercept)	x
107.44	10.93

Prediction

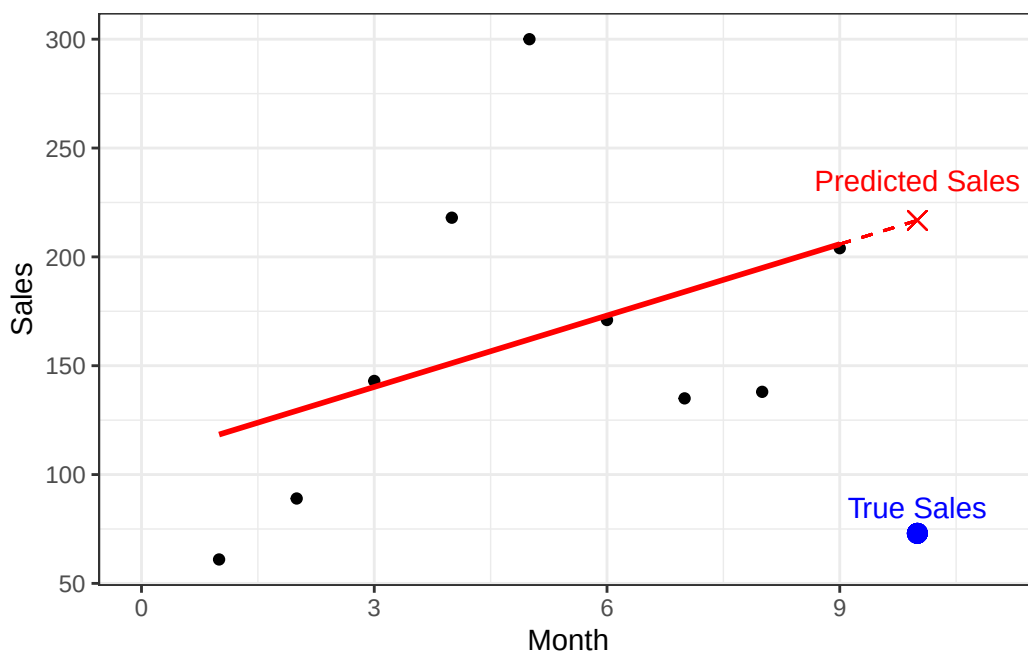
- This gives us a fitted model

$$\hat{f}(X_i) = \hat{\alpha} + \hat{\beta}X_i$$

- We could just evaluate this when $X_i = 10$ for the next month

$$\hat{f}(X_i = 10) = 107.44 + 10.93 \times 10 = 216.74$$

-
- Is this a good prediction?



Measuring Prediction

- We could compare the estimate to the true value, in terms of squared difference, a similar quantity to that used to fit the regression model initially
- For new data x_{new}, y_{new} , which was not used to fit the model, compute

$$\text{Prediction Error} = (y_{new} - \hat{f}(x_{new}))^2$$

$$\text{Prediction Error} = (y_{new} - \hat{f}(x_{new}))^2$$

- For this problem

```
alpha_hat <- as.numeric(coef(fit)[1])  
beta_hat <- as.numeric(coef(fit)[2])  
alpha_hat
```

```
[1] 107.4444
```

```
beta_hat
```

```
[1] 10.93333
```

```
y_hat <- alpha_hat + beta_hat * 10  
Error <- (y_hat - counts[10])^2  
Error
```

```
[1] 20672.05
```

- This is just a number, can't do anything with just a single number

Evaluating Predictions

- Come back to this after (very briefly) introducing an alternative method for doing prediction

K-Nearest Neighbours

Other prediction methods

- We have seen one initial method for predicting a numeric quantity, linear regression
- This is a simple but essential data science method
- We will now see another key method, which is quite different

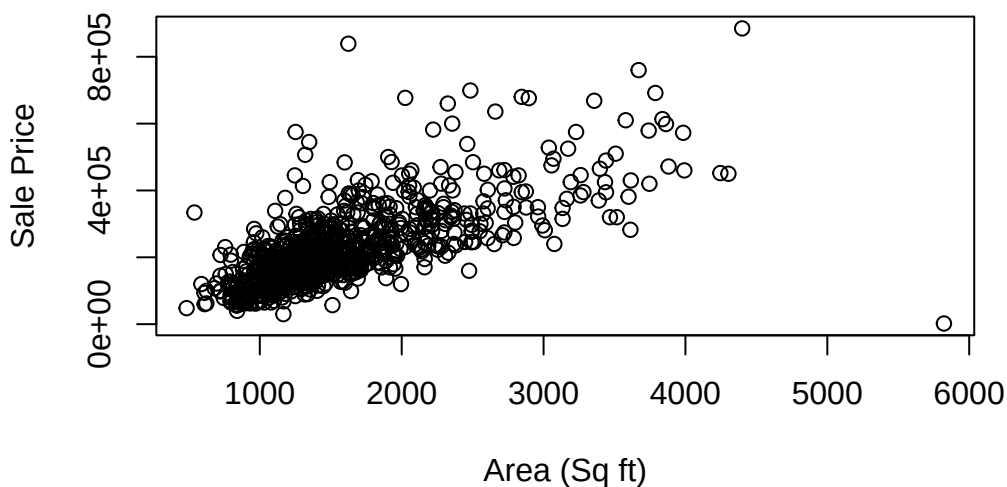
A Quick Note

- For the previous example, we wanted to predict the sales for the next month only, when $x_{new} = x + 1$
- This is kind of a special case, we want our method to be able to predict at any value of x normally
- We will see this in our next example and in all future examples

House Price Data

- Here we will use data about the sale price of ~1000 houses sold in California in 2008

```
house_sales <- read.csv("house_prices.csv")  
plot(x = house_sales$area, y = house_sales$price, xlab = "Area (Sq ft)",  
     ylab = "Sale Price")
```



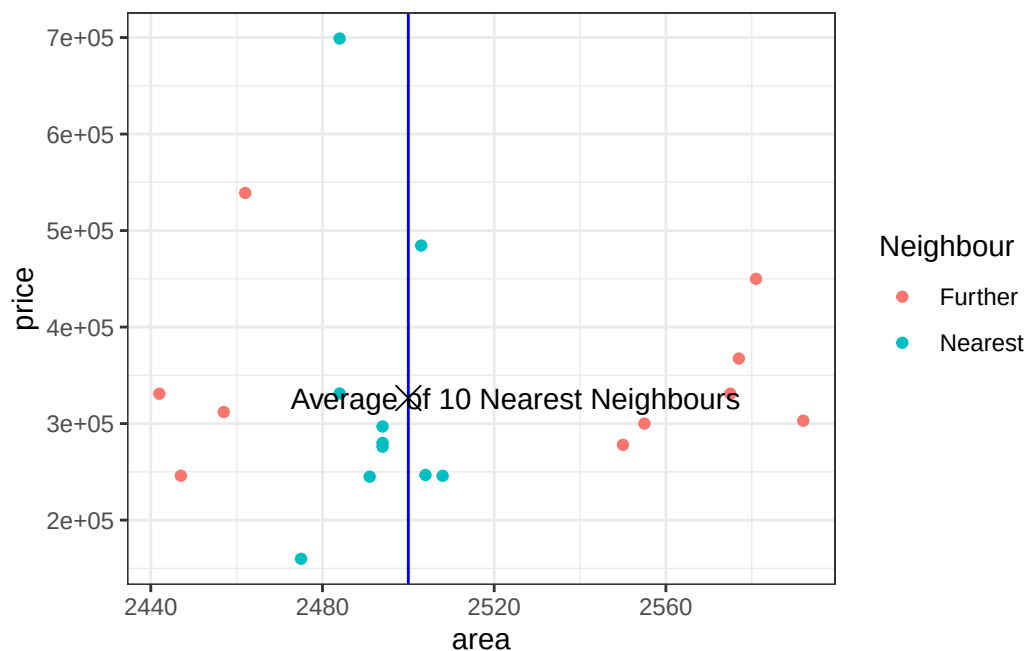
A Different Prediction Approach

- Suppose we want to predict the sale price of a new house (not in this dataset) which has an area of 2500 sq ft?
- We could do this with regression, find the value of $\hat{\alpha} + \hat{\beta} \times 2500$
- This requires the assumptions of a linear regression
- Instead, lets do something very simple

Neighbors

- Find the 10 rows in our dataset with x value “closest” to the one we want to predict at
- Get the average of Y for these 10 rows
- Use that as the prediction for a new house with $X = 2500$

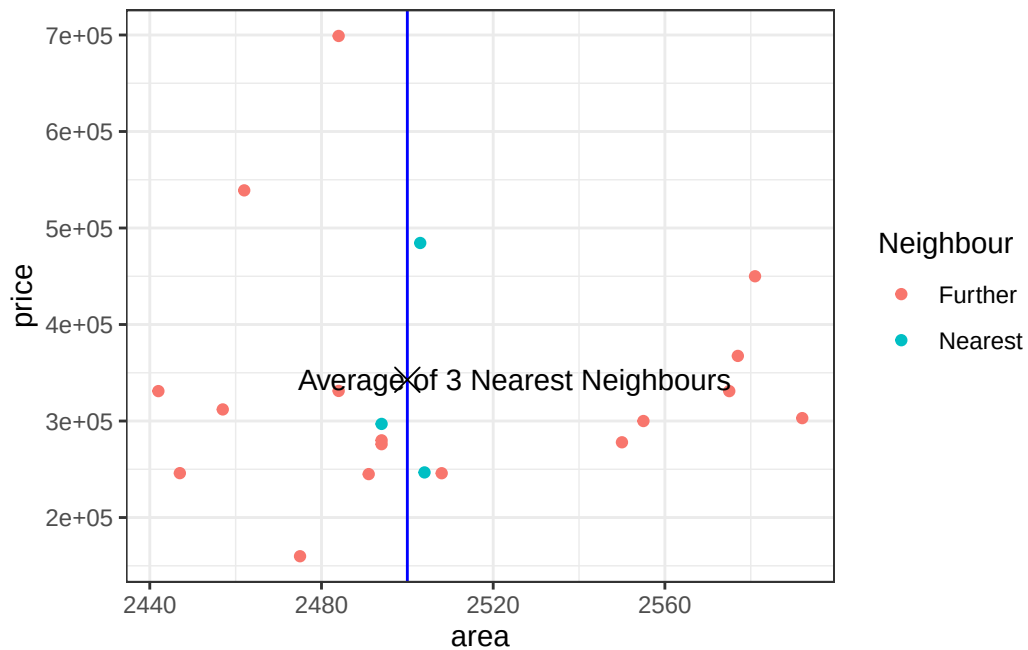
What this looks like



Notes

- How do we decide what points are closest?
- Squared distance, $(X - 2500)^2$
- Would this work if we had instead chosen the 3 nearest points?

-
- End up with a very similar estimate (for this example)



K-Nearest Neighbours

- This is the K in the name!
- How many neighbours do we use in our model?
- What is the right value of K for this problem?
- The solution to this is the same as the one needed to compare our linear regression model from before

Comparing Prediction Methods

Fitting a Model to Data

- Think back to how we fit linear regression
 - We find the model which minimizes the sum of squared errors on our data!
- This gives us the best possible regression model for our data
- But we want to see if its the best model for new/different data
- Several ways to do this

Training and Test Data

- Suppose we were trying to see how good our regression model is at predicting house prices here
- If this is all the data we have, we need to find a way to get “new” data
- The way to do this is to split our dataset into:
 - A training set
 - A test set

-
- Estimate our model using the training data
 - See how good it does at predicting the test data
 - Because we didn’t use the test data for fitting the model, this is predicting on new data
 - This is the standard way to evaluate models used for prediction
 - We just need some metric for performance on the test data

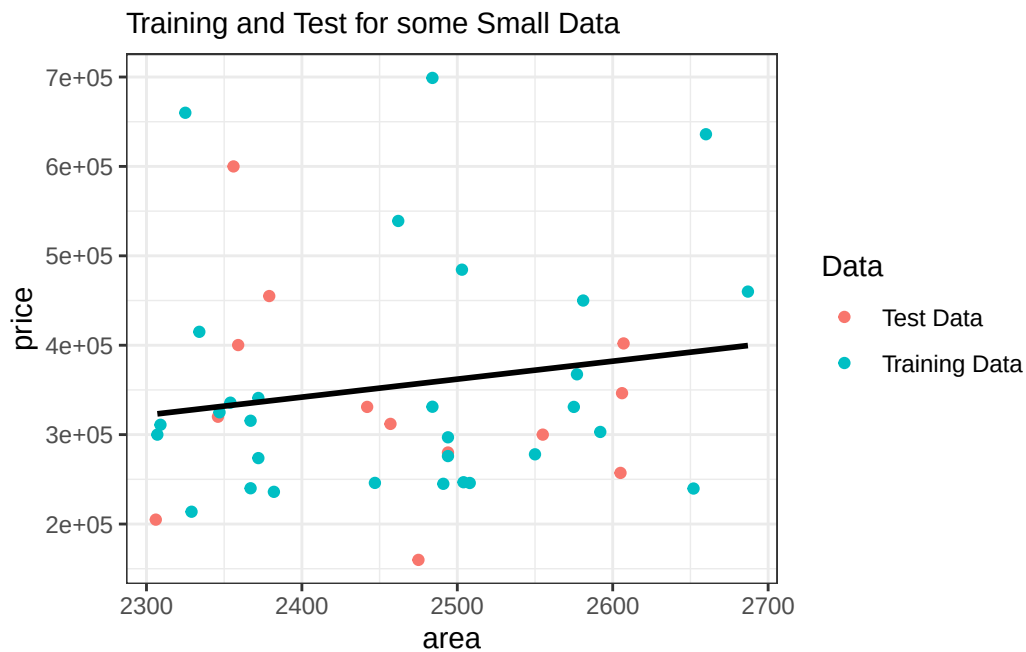
Test Data

- The test data has to be “new” data
- Data that was not used to estimate the model initially
- Otherwise we are cheating, won’t really tell us how good the model is at predicting
 - Known as overfitting

The Exact Procedure for Linear Regression

- We have data (X_i, Y_i)
- We randomly put 70% of this data in the training set and 30% in the test set (the proportions can vary)
- We estimate $\hat{\alpha}$ and $\hat{\beta}$ using **only the training data**
- We see how good the model is at predicting Y in the test data

What this looks like



Measuring Performance

- We need a measure of “performance” on the test data
- For a single point earlier we considered

$$\text{Prediction Error} = (y_{new} - \hat{f}(x_{new}))^2$$

- We need something like this, for all data in the test set
- Average it across the data in the test set

RMSPE

For (X_i^{test}, Y_i^{test}) in the test set we will compute

$$RSMPE = \sqrt{\frac{1}{n_{test}} \sum_{i=1}^{n_{test}} (Y_i^{test} - \hat{f}(X_i^{test}))^2}$$

- Here $\hat{\alpha}$ and $\hat{\beta}$ are computed from the training data!
- Then $\hat{f}(X_i^{test}) = \hat{\alpha} + \hat{\beta}X_i^{test} = \hat{Y}_i^{test}$
- The point on the fitted line, for the X in the test set
- This is the Root Mean Squared Predicted Error

Computing this

- Let’s split this house price data into a training and test split and compute the RMSPE

```
set.seed(240)
prop_train <- 0.7
prop_test <- 1 - prop_train

n_data <- nrow(house_sales)

train_test <- sample(c("Train", "Test"),
                    size = n_data, replace = TRUE,
                    prob = c(prop_train, prop_test))
head(train_test)
```

```
[1] "Train" "Test"  "Train" "Train" "Train" "Train"
```

```
head(train_test == "Train")
```

```
[1] TRUE FALSE TRUE TRUE TRUE TRUE
```

Splitting the data

```
train_data <- house_sales[train_test == "Train", ]  
test_data <- house_sales[train_test == "Test",]  
  
nrow(train_data)/n_data
```

```
[1] 0.7149877
```

```
nrow(test_data)/n_data
```

```
[1] 0.2850123
```

- `train_test` a vector indicating which rows are in which, then we subset using this to separate the data into the two subsets

Fit to Training Data

```
fit_train <- lm(price ~ area, data = train_data)  
fit_train
```

Call:

```
lm(formula = price ~ area, data = train_data)
```

Coefficients:

(Intercept)	area
30894.1	124.7

- These estimates will change if we have a different training set

Exercise

- Rerun the code to create the training and test sets, using a different seed (which will lead to a different sample). Confirm the estimates of α and β change when you do.

Solution

Getting Predictions

- Now we need to get predictions for Y for our test data
- We can use the R function `predict` to do this

```
est_Y <- predict(fit_train, newdata = test_data)
head(est_Y)
```

```
      2      13      14      16      19      21
176446.0 139527.9 158111.7 172330.1 104231.3 170334.5
```

- These are our estimates of $\hat{Y}_i^{test}$
- `predict` fits the model `fit_train`, using the X values from `test_data` (i.e, it knows X is area)

Computing RMSPE

- We need to compute

$$RMSPE = \sqrt{\frac{1}{n_{test}} \sum_{i=1}^{n_{test}} (Y_i^{test} - \hat{f}(X_i^{test}))^2}$$

- This is the same as

$$RMSPE = \sqrt{\frac{1}{n_{test}} \sum_{i=1}^{n_{test}} (Y_i^{test} - \hat{Y}_i^{test})^2}$$

-
- We just computed `est_Y` and can get Y_i^{test} from `test_data$price`

```
RMSPE <- sqrt( mean( (est_Y - test_data$price) ^2) )  
RMSPE
```

```
[1] 76317.91
```

- We have no other model to compare this to, can't really say if it's good or bad

Fitting KNN

- We will briefly fit KNN to this data next with $K = 5$
- We'll spend more time on this next week, but for now will just see how to do it

```
library(caret)  
  
fit_knn <- knnreg(price ~ area, data = train_data, k = 5)  
est_Y_knn <- predict(fit_knn, test_data)  
head(est_Y_knn)
```

```
[1] 135550.0 103313.2 144102.8 144098.0 103723.7 152950.0
```

Compute RMSPE

```
RMSPE_knn <- sqrt( mean( (est_Y_knn - test_data$price) ^2) )  
RMSPE_knn
```

```
[1] 82518.13
```

- This is larger than the RMSPE for linear regression, indicates that linear regression can better predict house prices using the area, for this data
- Compared to KNN with $K = 5$. But why $K = 5$?

Exercise

- Refit KNN with $K = 10$
- What is the RMSPE for this model on the test data?

Solution

Caveat

- This is a very brief/simplistic introduction to evaluating prediction models
- There are lots of important considerations we're avoiding
- If you take Stat 452 you'll spend a lot more time on this!

Choosing K

- We get different RMSPE for different choices of K ...
- How do we know which one is best?
- Is there a way to find the “best” value of K ?

Next Time

- We will talk in more detail about the KNN model
- How to choose the optimal value of K
- How to use KNN for another key data science task, classification